

L03. Introduction to Dynamic Programming

*Lecturer: Munther Dahleh**Scribe: Jinhee Won*

1 Introduction

In the previous lecture, we defined how to model a dynamic process. We discussed the state, the transition properties, and the cost. Today, we will continue discussing cost and introduce the Bellman equation, a fundamental building block of dynamic programming and reinforcement learning.

So, let's start by defining dynamic programming. Dynamic programming is a general mathematical framework for sequential decision-making under uncertainty. We consider a system that evolves over time according to some given rules, and our goal is to choose a sequence of actions that minimizes a certain additive cost.

“Dynamic programming: a name not even a Congressman could object to.” —Richard Bellman

2 Terminology

Our model consists of the state, the dynamics, the stage cost, the controls, and a model of uncertainty. We also started discussing feedback policies. A feedback policy is a strategy that tells us which action to take at each given state. Once we specify the cost that we are trying to minimize, our goal is to find the feedback policy that minimizes that cost.

Aside

Here, we can intuitively start to see why being Markovian is important. It allows us to define a strategy that depends only on the current state, without having to worry about the entire history of the process. For example, suppose you are playing chess, and your state is defined as the positions of the pieces: you move your knight forward and then back, and your opponent does the same, so that you return to exactly the same state. The strategy you apply should remain the same because all the relevant information is already captured by the state.

2.1 Review

Beyond the queueing problem that we discussed previously, this modeling formulation can be applied to a broad range of problems. As another example, consider a stopping-time problem involving a call option.

Suppose that S_t is the price of the underlying asset at time t . Its price evolves according to

$$S_{t+1} = S_t + W_t, \quad S_0 = s_0,$$

where W_t is a random price change. For example, we may have

$$W_t \in \{-1, 0, +1\}$$

or

$$W_t \sim \mathcal{N}(0, \sigma^2).$$

The problem has strike price p , time horizon T , and discount factor β . At each time, our action is

$$a_t \in \{0, 1\},$$

where $a_t = 0$ means that we hold the option and $a_t = 1$ means that we exercise it. Because the option can be exercised at most once, we impose the constraint

$$\sum_{t=0}^T a_t \leq 1.$$

The objective is to maximize the expected reward

$$J = \mathbb{E} \left[\sum_{t=0}^T a_t (S_t - p) \right].$$

This example may initially seem different from the queueing problem as it seems there are no intermediate rewards; if we hold the option, we receive nothing at that time. However, we can actually just define the immediate reward as zero in these intermediate states, with a reward realization only when we exercise the option.

Aside

When is a greedy solution and dynamic programming solution equivalent? They are equivalent when choosing the action with the best immediate reward never sacrifices any future value. In terms of the Bellman equation, which we discuss later in this lecture,

$$V_t^*(s) = \max_{a \in A} \{r_t(s, a) + \mathbb{E} [V_{t+1}^*(s_{t+1}) \mid s_t = s, a_t = a]\}$$

a greedy policy is optimal if an action that maximizes the immediate reward $r_t(s, a)$ also maximizes the entire expression for every state and time.

2.2 Discounting

We further define two types of problems: finite-horizon and infinite-horizon problems. In a finite-horizon problem, the process terminates after some fixed time N . In an infinite-horizon problem, the process can continue forever.

This creates an immediate issue. Suppose that we receive a positive reward at every time step. If the process never terminates, then the total reward will approach infinity. There is no natural reason for the process to stop, so the objective can not be computed.

To handle this issue, we introduce a discount factor γ . In the the discounted infinite-horizon problem,

$$\min_{\pi} \lim_{N \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^{N-1} \gamma^t c_t(s_t, a_t, w_t) \right],$$

such that

$$s_{t+1} = f_t(s_t, a_t, w_t), \quad a_t = \mu_t(s_t) \in \mathcal{A}_t(s_t), \quad w_t \sim \mathbb{P}(\cdot \mid s_t, a_t).$$

For an infinite-horizon problem, we take $0 \leq \gamma < 1$ as we wish for our value function to be bounded (given stage costs are bounded; proof is left to the reader). However, within this range, the gamma value is very problem dependent. When γ is close to 1, future costs are weighted almost as heavily as immediate costs. When γ is close to 0, the objective places most of its weight on the immediate cost.

Further, note we can also have a discount on the finite-horizon problem to control the weight of future costs:

$$\min_{\pi} \mathbb{E} \left[\gamma^N c_N(s_N) + \sum_{t=0}^{N-1} \gamma^t c_t(s_t, a_t, w_t) \right].$$

Aside

Another approach is to consider the average cost per time step:

$$\min_{\pi} \lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E} \left[\sum_{t=0}^{N-1} c_t(s_t, a_t, w_t) \right].$$

This formulation does not require discounting and can still produce a finite objective even when the total cost is infinite. We will not cover average-cost problems in detail in this course.

2.3 Stationarity

We will define the model so that the system is stationary. Here, “time” does not necessarily refer to physical or calendar time; t simply indexes the steps of the process. Depending on the problem, one step might represent a second, a day, a customer arrival, a financial transaction, or a move in a game. It is simply the unit that takes us from one state to the next.

Informally, we want a state on September 17 to mean the same thing as that same state on September 18, or even 1,000 years in the future. In other words, the rules governing the transition from one step to the next should not change merely because the step occurs later. Formally, stationarity means that the dynamics, stage cost, action constraints, and model of uncertainty do not depend explicitly on t . Similarly, a stationary policy uses the same decision rule at every step. If physical time does affect the system, we can actually include it as part of the state while continuing to use t simply as an index for the steps of the process.

3 Solving Dynamic Programming Problems

Recall that

$$V_N^{\pi}(s_0) = \mathbb{E}^{\pi} \left[c_N(s_N) + \sum_{t=0}^{N-1} c_t(s_t, \mu_t(s_t), w_t) \right].$$

Our goal is to find the feedback policy that minimizes this cost:

$$V^*(s_0) = \min_{\pi} V^{\pi}(s_0), \quad \pi \in \arg \min_{\pi} V^{\pi}(s_0).$$

3.1 Deterministic Problems

We begin with the simplest case: a deterministic, finite-horizon problem. Because the system is deterministic, once we apply a particular control at a given state, we know exactly which state comes next. A classic example is a linear system.

For any fixed sequence of actions, the entire state trajectory is predetermined. In principle, we could enumerate all possible action sequences, calculate the cost of each resulting trajectory, and then select the best one. In certain simple cases, we may even be able to find a closed-form solution. However, this approach can be boring and slightly impractical.

Instead, we solve the problem by working backward. At the terminal state, there are no actions left to choose, so the value function of the optimal policy is

$$V_N^*(s) = c_N(s).$$

Now suppose that we are in one state prior at time $N - 1$. If we choose action a , we incur the immediate cost $c_{N-1}(s, a)$ and transition to the terminal state $f_{N-1}(s, a)$. Thus,

$$V_{N-1}^*(s) = \min_{a \in \mathcal{A}_{N-1}(s)} \{c_{N-1}(s, a) + c_N(f_{N-1}(s, a))\}.$$

Once we have computed these values, we move backward to time $N - 2$. More generally, the optimal cost-to-go becomes

$$V_t^*(s) = \min_{a \in \mathcal{A}_t(s)} \{c_t(s, a) + V_{t+1}^*(f_t(s, a))\}.$$

Aside

Many of you have probably seen dynamic programming problems on LeetCode or in algorithms courses. We robustly formulate this framework as it is critical to reinforcement learning and optimal control. Further, thinking in terms of states, actions, transitions, and costs can make many dynamic programming problems much easier to approach (and also code).

3.2 Stochastic Problems

The stochastic, finite-horizon case can become a little more complicated. Given a state and action, we may have that the next state is non-deterministic and instead follows a probability distribution. We can show that the optimal value function behaves the same way; but with slightly more math that requires careful review.

3.2.1 First, a fixed policy.

First, fix a feedback policy

$$\pi = \{\mu_0, \dots, \mu_{N-1}\},$$

where μ_t is the decision rule at time t : given state s , it selects an action $\mu_t(s) \in \mathcal{A}_t(s)$. Define

$$G_t = \sum_{\tau=t}^N c_{\tau}(s_{\tau}, \mu_{\tau}(s_{\tau}))$$

This is simply the total cost from time t to the terminal time N under the fixed policy π . Its expected value at the current state is

$$V_t^{\pi}(s) = \mathbb{E}^{\pi}[G_t \mid s_t = s].$$

As before, we work backward. At the terminal time,

$$V_N^{\pi}(s) = c_N(s).$$

In the steps prior, we obtain

$$\begin{aligned} V_t^{\pi}(s) &= c_t(s, \mu_t(s)) + \mathbb{E}^{\pi}[G_{t+1} \mid s_t = s] \\ &= c_t(s, a) + \mathbb{E}^{\pi}[\mathbb{E}^{\pi}[G_{t+1} \mid s_t = s, s_{t+1}] \mid s_t = s] \\ &= c_t(s, a) + \sum_{s'} p_t(s' \mid s, a) \mathbb{E}^{\pi}[G_{t+1} \mid s_{t+1} = s'] \\ &= c_t(s, a) + \sum_{s'} p_t(s' \mid s, a) V_{t+1}^{\pi}(s'). \end{aligned}$$

From the first to the second line, we condition on the next state and take the expectation over all of its possible values. To get to the third line, we use the Markov property: under our fixed feedback policy, once we know s_{t+1} , knowing s_t provides no additional information about the future cost G_{t+1} . Finally, the inner expectation is exactly the definition of $V_{t+1}^{\pi}(s')$.

3.2.2 Then, finding the optimal policy.

Now, we have to consider the optimal policy, which minimizes this value

$$V_t^*(s) = \min_{\pi \in \Pi} V_t^{\pi}(s).$$

We can again work backward. At the time $N-1$, this is straightforward: we choose the action that minimizes the immediate cost plus the expected terminal cost:

$$\begin{aligned} V_{N-1}^*(s) &= \min_{\mu_{N-1}} \mathbb{E} \left[c_{N-1}(s, \mu_{N-1}(s)) + c_N(s_N) \mid s_{N-1} = s \right] \\ &= \min_{\mu_{N-1}} \left\{ c_{N-1}(s, \mu_{N-1}(s)) + \sum_{s'} p_{N-1}(s' \mid s, \mu_{N-1}(s)) c_N(s') \right\} \\ &= \min_{\mu_{N-1}} \left\{ c_{N-1}(s, \mu_{N-1}(s)) + \sum_{s'} p_{N-1}(s' \mid s, \mu_{N-1}(s)) V_N^*(s') \right\}. \end{aligned}$$

At time $N-2$, things become a little trickier because we must optimize over two decision rules:

$$\begin{aligned}
V_{N-2}^*(s) &= \min_{\mu_{N-2}, \mu_{N-1}} \mathbb{E} \left[c_{N-2}(s, \mu_{N-2}(s)) + c_{N-1}(s_{N-1}, \mu_{N-1}(s_{N-1})) + c_N(s_N) \mid s_{N-2} = s \right] \\
&= \min_{\mu_{N-2}, \mu_{N-1}} \left\{ c_{N-2}(s, \mu_{N-2}(s)) + \sum_{s'} p_{N-2}(s' \mid s, \mu_{N-2}(s)) V_{N-1}^\pi(s') \right\} \\
&= \min_{\mu_{N-2}} \left\{ c_{N-2}(s, \mu_{N-2}(s)) + \min_{\mu_{N-1}} \sum_{s'} p_{N-2}(s' \mid s, \mu_{N-2}(s)) V_{N-1}^\pi(s') \right\}.
\end{aligned}$$

We would like to move the inner minimization inside the sum so that we can replace the remaining cost at each next state with its optimal value

$$\min_{\mu_{N-1}} V_{N-1}^\pi(s') = V_{N-1}^*(s'),$$

which we have already computed.

So the question is: can we switch averaging and minimization?

Here, we can, because our policy tells us what to do in each possible next state. We do not have to choose the same action everywhere: we can choose whichever action is best for the state we actually reach. Thus,

$$V_{N-2}^*(s) = \min_{\mu_{N-2}} \left\{ c_{N-2}(s, \mu_{N-2}(s)) + \sum_{s'} p_{N-2}(s' \mid s, \mu_{N-2}(s)) V_{N-1}^*(s') \right\}.$$

Finally, repeating this argument at each earlier time gives us the Bellman equation:

$$V_t^*(s) = \min_{a \in \mathcal{A}_t(s)} \left\{ c_t(s, a) + \sum_{s'} p_t(s' \mid s, a) V_{t+1}^*(s') \right\}.$$

Aside

It is not a general rule that minimization and expectation can always be interchanged. In this case this holds, and we can formalize as follows.

Let $F(s, s', \mu_{N-1}(s'))$ denote the remaining cost, with the current state s and action fixed. For any decision rule μ_{N-1} ,

$$\mathbb{E}[F(s, s', \mu_{N-1}(s'))] \geq \mathbb{E} \left[\min_{\mu_{N-1}} F(s, s', \mu_{N-1}(s')) \right].$$

Since this holds for every rule, it also holds after minimizing the left-hand side,

$$\min_{\mu_{N-1}} \mathbb{E}[F(s, s', \mu_{N-1}(s'))] \geq \mathbb{E} \left[\min_{\mu_{N-1}} F(s, s', \mu_{N-1}(s')) \right].$$

For the reverse inequality, first note the best rule μ_{N-1}^* cannot cost more than one specific rule you choose,

$$\min_{\mu_{N-1}} \mathbb{E}[F(s, s', \mu_{N-1}(s'))] \leq \mathbb{E}[F(s, s', \mu_{N-1}^*(s'))].$$

Now assume that a minimizing action exists at each state. Since the state space is finite, we can choose μ_{N-1}^* to select a minimizing action at every next state s' . For this choice, the cost equals its pointwise

minimum, so

$$\mathbb{E} [F(s, s', \mu_{N-1}^*(s'))] = \mathbb{E} \left[\min_{\mu_{N-1}} F(s, s', \mu_{N-1}(s')) \right].$$

Combining these proves equality.

3.3 Downstream Implications

With all of this context, we can revisit the inventory problem we discussed during recitation. We can now formalize why the value function can be written in this form under the optimal policy. This ultimately completes our solution to the constrained inventory control problem and allows us to prove the existence of an optimal threshold using the convexity argument.

In future lectures and in the homework, you will explore further examples of this approach and investigate where, when, and how these applications can be extended, modified, or shown to break down.